Reproducing Segment Policy Optimization on GSM8K

A tutorial on RL for language models, told through one audited reproduction

Alex Towell

October 2026 Paper: Guo, Xu, Liu, Ye, Qiu, arXiv:2505.23564. Code: AIFrameResearch/SPO.

In one sentence

Rerunning the authors' code on one A100 took a math model from 39.8% to 58.2% on GSM8K, a little above the paper's 56.7%, with the comparison point fixed before training.

Reproduce a known result before trusting a new one.

The situation. We want to train a model with reinforcement learning (RL) on a task where nobody has published numbers. If it fails, we will not know whether the method, the code, or the measurement is at fault.

The move. First rerun a published RL result whose answers can be checked automatically. Every part of the pipeline gets tested against a known target.

What a reproduction buys

- A tested evaluator (score the authors' released model).
- A tested training loop (retrain and recover the score).
- A list of the places where things silently go wrong.

This deck follows the written walkthrough at queelius.github.io/spo-gsm8k-reproduction; every number has a receipt in the repository's docs/evidence folder.

The task: grade-school math with a checkable final answer.

Question (GSM8K test, item 0). Janet's ducks lay 16 eggs per day. She eats three for breakfast every morning and bakes muffins for her friends every day with four. She sells the remainder at the farmers' market daily for $2 per fresh duck egg. How much in dollars does she make every day at the farmers' market?

Gold solution.
Janet sells $16 - 3 - 4 = 9$ duck eggs a day.
She makes $9 \times 2 = 18$ dollars every day.
18

How an answer is scored

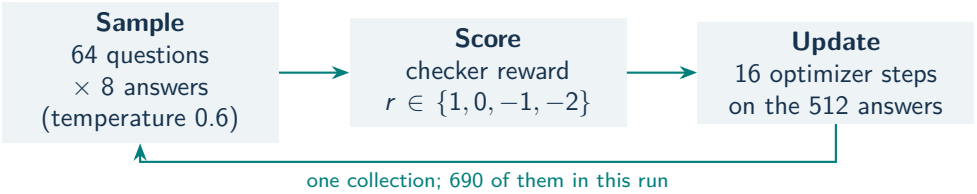
Take the text after the last ####, strip commas, compare as a string to the gold answer. Correct $\rightarrow$ reward 1; wrong $\rightarrow$ 0; no marker or cut off $\rightarrow -1$.

Splits used: 7,100 train / 373 validation / 1,319 test.

Metric: mean pass@1 over 16 sampled answers per test question (21,104 answers).

Cobbe et al. 2021 (GSM8K). The checker is the authors' own; it is exact string matching, so 1600 against 1,600 scores zero.

RL for a language model: sample, score, update.



Policy $\pi_\theta(y \mid x)$: the model's probability of answer y to prompt x, one token at a time.

Advantage A_t: how much better token t was than expected. Positive advantage raises its probability.

PPO clipping. With ratio $\rho_t = \pi_\theta / \pi_{\text{old}}$, the loss is $-\min(\rho_t A_t,\ \text{clip}(\rho_t, 0.8, 1.2)\, A_t)$, so one update cannot move far from the sampler. A small KL term (10^{-4}) keeps the policy near the starting model.

Schulman et al. 2017 (PPO). The guard that zeroes a batch whose mean ratio exceeds 10 returns on slide 10.

The problem: one number at the end has to pay 100 to 300 tokens.



Trajectory level

Cheap, no critic. But a wrong answer with a good first half gets no internal distinction, and a group of 8 that all agree gives zero signal.

Token level (PPO + critic)

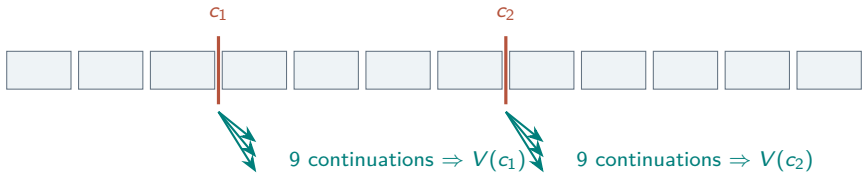
A value for every token, from a second model trained on noisy targets. Most informative in principle; the estimates are often inaccurate in practice.

Segment level (SPO)

Cut the answer at a few points, estimate the value at each cut by sampling, give each segment one advantage. VinePPO does this at every line; SPO cuts fewer, smarter.

Shao et al. 2024 (GRPO); Kazemnejad et al. 2024 (VinePPO). The SPO code base is a fork of VinePPO's.

SPO-chain: cut where the model was unsure, value each cut by sampling.



1. Partition. Mark tokens the sampler gave $p_t < 0.9$; shift each mark back one token; keep every 5th mark (the *interval*). About 3 cutpoints per answer here.

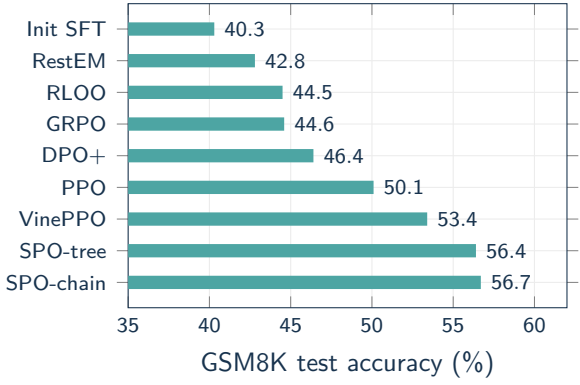
2. Value at a cut. $V(c_k) = \frac{1}{9} \sum_{j=1}^{9} r(x, y_{\leq c_k} \oplus z_j)$: the chance of finishing correctly from this prefix. No critic.

3. Segment advantage. $A_k = V(c_k) - V(c_{k-1})$ for the tokens between two cuts; the last segment gets $r - V(c_K)$; the first gets 0 in the public code.

4. Update. Mask out confident tokens ($p_t > 0.9$), whiten the advantages, then the PPO-clip step of the previous slide.

As implemented at the pinned commit. Cost: about 14,600 continuation samples per 512 answers.

What the paper reports, and what we set out to match.



Setting. RhoMath 1.1B, already supervised-fine-tuned on GSM8K, then RL.

Target. SPO-chain, 56.7%, which the audit traced to checkpoint 690 of a 1,000-iteration schedule in the authors' logs.

Caveat the paper's own caption states: the baseline bars are inherited from the VinePPO paper, not rerun.

Figure 3 of the official README (RhoMath 1.1B, GSM8K). Values read from the figure; 56.7 confirmed from the authors' W&B log (0.56724).

What we ran: the authors' code, one GPU, a comparison point fixed in advance.

Code	AIFrameResearch/SPO at commit 6c5f947, unmodified
Start	realtreetune/rho-1b-sft-GSM8K (the authors' SFT model)
Recipe	recovered from the authors' logs: seed 42, LR 10^{-6}, 64 questions $\times$ 8 answers, 9 continuations, interval 5
Hardware	one A100 40 GB, about 51 hours
Endpoint	collection 690 / update 11,040, declared before training

Why fix the endpoint first

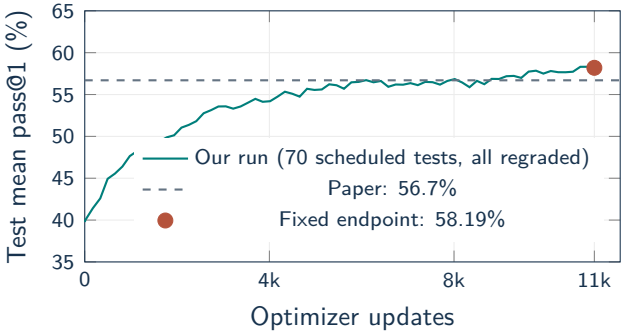
The paper's number is one checkpoint of a long schedule, and nothing records why that one. Picking our best test checkpoint afterwards would make the comparison meaningless.

Two separate checks

Scoring the authors' released model (56.69% here vs 56.72% in their log) tests the *evaluator*. Training our own model tests *learning*. Only the second is the result.

docs/protocol-audit.md; docs/evidence/author-reference-*.json; environment lock env/requirements-spo-6c5f947.txt.

Result: our training recovered the paper's level, at a pre-declared endpoint.



Matched SFT start	39.77%
Paper	56.70%
Ours, endpoint	**58.19%**
Ours, best intermediate	58.32%

12,281 of 21,104 answers correct.

Gain over the matched start: +18.4 points.

One seed, one task: a performance-level reproduction, not statistical equivalence.

docs/evidence/final690-audit-20261014.json; slides/learning-curve.csv with curve-provenance.json mapping each point to its receipt and hash.

The first attempt collapsed; the diagnosis is the most useful part.

What we saw. Test accuracy rose to 50.5% at 4,336 updates, fell to 22.2% at 7,856, and the process crashed at step 7,995.

Was it the scorer? No: an independent CPU regrade of 66,048 saved answers matched every metric. The model really got worse (96 questions went from 16/16 to 0/16).

Why the crash? Four batches in a row hit the PPO ratio guard (mean ratios 95, 37, 153, 10), were zeroed, and the BF16 optimizer asserted on a gradient norm of exactly zero.

The input mismatch

Generation requests started with the BOS token; the training-side tokenizer omitted it. Answers were sampled under one input and scored under another. Adding BOS shifted the late model's log-likelihood by $+54$ nats on average (-0.1 for the SFT model).

Fixes, each isolated and hash-pinned: a BOS adapter, a finite-zero-norm adapter, a fixed-endpoint hook; then a restart from the original SFT weights.

docs/diagnosis-and-restart-20261012.md. The mismatch is a supported hypothesis for the decline, not a demonstrated cause: no ablation was run.

Deviations from the paper, stated plainly.

Item	What differs and why
Evaluation top-p	Paper says 0.9; the evaluation template omits the argument, so 1.0 applies, in the authors' historical pipeline too. Kept at 1.0 for an exact reference comparison.
BOS token	Prepended during training by our adapter; the public source has it commented out. Whether the authors' run added it is unknown.
Zero gradient norm	Accepted as finite (DeepSpeed asserts > 0); NaN and Inf still rejected.
Config vs code	Recipe recovered from the authors' logs, not a recorded git revision; README defaults differ from the historical run.
Scope	One seed, one task, no GRPO control, checkpoint copies on the same filesystem.

Full list with receipts: docs/protocol-audit.md and docs/final-result-20261014.md.

Lessons that transfer to any RL experiment.

- **Fix the comparison point before training**, or you will select the best test checkpoint without noticing.
- **Score a known model first.** Seven minutes on the authors' checkpoint separated evaluator bugs from training bugs for a week.
- **Report the gain from a matched baseline** under one scoring rule: 39.8 $\rightarrow$ 58.2 is the result, not 58.2.
- **Check that the tokens you reward are the tokens you scored**, BOS included.
- **Regrade saved answers independently** of the training code; it is cheap and rules out scoring bugs in minutes.
- **Read the safeguards.** A guard that "skips" a batch produced a zero gradient and crashed the optimizer.
- **Preserve failures.** The collapsed run's artifacts made the diagnosis possible.

Every item above corresponds to a receipt in docs/evidence; the walkthrough's section 11 lists them.

Reproduce it yourself.

1. **Environment.** Python 3.10 from `env/requirements-spo-6c5f947.txt`; clone AIFrameResearch/SPO at commit 6c5f947 and leave it unmodified.
2. **Assets.** Dataset archive and the two Hugging Face models at the pinned revisions; verify the hashes in `docs/evidence/`.
3. **Reference checks.** Run the recorded argv in `author-reference-command.json`; regrade; expect 0.5669 and 0.3977.
4. **Train.** `code/launch_bos_only.py -configs fresh_bos_only.jsonnet run_iteration_loop` with the environment variables in `restart-20261012.json`; run the CPU tests first.
5. **Watch, regrade, preserve.** `code/review_monitor.py`; regrade each scheduled evaluation; after the endpoint hook, copy and hash the 690 checkpoint.

Copy commands from the receipts rather than retyping them; the walkthrough's section 12 names each one.
Repository: github.com/queelius/spo-gsm8k-reproduction (MIT; the authors' code is MIT).

Appendix: glossary.

Advantage	value achieved minus value expected; the per-token learning signal.
BOS / EOS	beginning- and end-of-sequence tokens a tokenizer may add.
Collection	64 questions $\times$ 8 answers plus Monte Carlo sampling, then 16 updates.
Cutpoint	a response position chosen for a value estimate.
GRPO	group-normalised trajectory-level advantages; no critic.
Mean pass@1	mean over questions of the fraction of 16 samples that are correct.
Monte Carlo value	mean reward of sampled continuations from a prefix.
PPO ratio	new-policy over sampling-policy probability of a token.
Probability mask	exclude tokens the sampler was already sure about ($p_t > 0.9$).
Segment	the tokens between two cutpoints.
SFT	supervised fine-tuning; the starting model was SFT'd on GSM8K.